

Soirée d'intégration : multi-star, drill-across et réel-vs-cible

-  4 juin 2026
-  19 h 00 – 22 h 00
-  Longueuil
-  **Temps estimé** : 105 min (~1.8 h)

OBJECTIF DE LA SÉANCE

Intégrer les étoiles indépendantes de NexaMart via les dimensions conformes. Construire 4 tables de faits, écrire des requêtes drill-across, et comparer le réel aux cibles budgétaires.

QUESTION DU CEO

« Le board peut-il voir ventes, retours, inventaire et budget dans une seule vue sans mentir ? »

CONTEXTE DU SOIR

NexaMart S07 : Le board peut-il voir ventes, retours, inventaire et budget

C'est la soirée d'intégration. Jusqu'à présent, chaque étoile existait indépendamment. Le CEO veut une vue consolidée : ventes réelles vs budget, taux de retour par catégorie, et niveaux d'inventaire. Tout doit passer par des dimensions conformes.

Prof. Carlos Denner dos Santos, Ph.D.

Département des systèmes d'information et méthodes quantitatives de gestion (SIMQG) - École de gestion
Université de Sherbrooke

Objectifs d'apprentissage

À LA FIN DE CETTE SÉANCE, VOUS SEREZ CAPABLE DE...

1. Concevoir plusieurs tables de faits partageant des dimensions conformes.
2. Écrire une requête drill-across correcte entre fact_sales et fact_returns.
3. Construire une vue réel-vs-budget sans joindre des faits entre elles.
4. Publier un bus matrix documentant la conformité entre vos tables de faits.

POINTS CLÉS

- 💡 Cette soirée intègre vos étoiles indépendantes en un entrepôt cohérent.
- 💡 Chaque jointure doit passer par des dimensions conformes — jamais fact-to-fact.
- 💡 Le bus matrix est le document le plus important de l'intégration.

IDÉES REÇUES À ÉVITER

⚠️ On peut joindre deux tables de faits directement

- ✓ Joindre fact_sales à fact_returns crée un produit cartésien. Le drill-across passe par les dimensions conformes.

⚠️ Budget et ventes ont le même grain

- ✓ fact_budget est mensuel par catégorie×magasin. fact_sales est transactionnel. Le drill-across exige une agrégation au grain commun.

PRIORITÉS DU SOIR

CORE

conformed dimensions

drill across

bus matrix

actual vs budget

STRETCH

mixed grain handling

Déroulement de la séance

STRUCTURE DE LA SÉANCE

Partie	Titre	Durée	Contenu
1	Multi-star theory + bus matrix	20 MIN	Dimensions conformes, drill-across, grains multiples
2	Sprint 1 : integration night	45 MIN	Charger 4 fact tables, vérifier conformité, écrire drill-across
3	Sprint 2 : actual-vs-budget + board report	40 MIN	Vue réel-vs-cible, rapport consolidé pour le board

OBJECTIF

Intégrer les étoiles indépendantes NexaMart via les dimensions conformes. Construire un drill-across entre fact_sales et fact_returns, une vue actual-vs-budget au grain commun, et répondre à la question du CEO avec évidence provenant d'au moins deux tables de faits.

LIVRABLE ATTENDU

Bus matrix + drill-across ventes x retours + vue actual-vs-budget + board brief.

ARTÉFACTS DU REPO

- ☐ answers/S07_executive_brief.md
- ☐ docs/bus-matrix.md
- ☐ sql/integration/s07-drill-across.sql
- ☐ sql/integration/s07-actual-vs-budget.sql
- ☐ sql/checks/s07-reconciliation.sql
- ☐ docs/board-briefs/s07-enterprise-view.md

CHECKLIST DE REMISE

- ☐ answers/S07_executive_brief.md
- ☐ docs/bus-matrix.md
- ☐ sql/integration/s07-drill-across.sql
- ☐ sql/integration/s07-actual-vs-budget.sql
- ☐ docs/board-briefs/s07-enterprise-view.md

MATÉRIAUX FOURNIS

- Données uniques auto-générées depuis le username GitHub (make generate)
- Makefile: make generate/load/check

EXERCICES

Exercice 00 — Construire les 3 tables de faits manquantes

30 min

Écrire et charger les 3 tables de faits introduites en S06 mais jamais construites (S06 était l'examen). Sans ces tables dans DuckDB, les exercices 0 à 3 échouent avec « Table not found ».

1. `make generate && make load`
2. `SHOW TABLES;`
3. *-- fact_sales doit être là. Les 3 autres manquent probablement.*
4. *-- ÉTAPE 1 : fact_budget (commencez ici – pas de SCD2)*
5. `DESCRIBE raw_fact_budget;`
6. *-- Créez sql/facts/fact_budget.sql avec :*
7. *-- CREATE OR REPLACE TABLE fact_budget AS*
8. *-- SELECT rb.budget_id, CAST(rb.budget_month AS DATE), rb.category,*
9. *-- st.store_key, CAST(rb.target_revenue AS DECIMAL(12,2)), CAST(rb.target_units AS INTEGER)*
10. *-- FROM raw_fact_budget rb JOIN dim_store st ON st.store_id = rb.store_id*
11. *-- ÉTAPE 2 : fact_inventory_snapshot*
12. `DESCRIBE raw_fact_inventory_snapshot;`
13. *-- Créez sql/facts/fact_inventory_snapshot.sql*
14. *-- Colonnes : snapshot_id, snapshot_date, product_key, store_key,*
15. *-- quantity_on_hand, quantity_on_order, reorder_point,*
16. *-- below_reorder (CASE WHEN qty < reorder THEN TRUE ELSE FALSE END)*
17. *-- Sources : raw_fact_inventory_snapshot JOIN dim_product ON product_id JOIN dim_store ON store_id*
18. *-- ÉTAPE 3 : fact_returns (résolution SCD2 – la plus complexe)*
19. `DESCRIBE raw_fact_returns;`

```

20. -- Créez sql/facts/fact_returns.sql
21. -- Colonnes : return_id, original_sale_line_id, return_date, product_key,
22. -- store_key, customer_key, return_quantity, refund_amount, return_reason
23. -- Indice SCD2 : ajoutez une CTE pour récupérer order_date depuis raw_fact_sales :
24. -- WITH sale_context AS (SELECT sale_line_id,
25. -- CAST(order_date AS DATE) AS original_order_date, customer_id FROM raw_fact_sales)
26. -- puis LEFT JOIN dim_customer ON customer_id
27. -- AND original_order_date BETWEEN effective_from AND effective_to
28. -- ÉTAPE 4 : make load et vérification
29. make load
30. SELECT COUNT(*) FROM fact_returns; -- doit être > 0
31. SELECT COUNT(*) FROM fact_inventory_snapshot; -- doit être > 0
32. SELECT COUNT(*) FROM fact_budget; -- doit être > 0
33. git add sql/facts/ && git commit -m 'S07 ex00 3 fact tables created and loaded'

```

⚠ Écueils fréquents

- fact_returns sans la CTE SCD2 : customer_key sera NULL pour tous les retours
- fact_budget avec un JOIN dim_product : raw_fact_budget n'a pas de product_id
- make generate inutilement relancé : make load suffit une fois les SQL écrits
- Oublier CAST pour les colonnes numériques : les types raw peuvent différer

Exercice 0 — Découverte du schéma

20 min

Comprendre le design de chacune des 4 tables de faits avant d'écrire une seule requête d'intégration. Vous devez articuler par écrit les contraintes de jointure avant les sprints.

```

1. make generate && make load (ou make check si déjà fait)
2. SELECT COUNT(*) FROM fact_sales; SELECT COUNT(*) FROM fact_returns;
3. SELECT COUNT(*) FROM fact_inventory_snapshot; SELECT COUNT(*) FROM fact_budget;
4. -- Toutes les tables doivent être non vides. Si une est vide : make generate puis make load.
5. DESCRIBE fact_sales; DESCRIBE fact_returns; DESCRIBE fact_inventory_snapshot; DESCRIBE fact_budget;
6. -- Répondez à ces 3 questions dans un bloc -- OBSERVATIONS : au début de votre fichier SQL :
7. -- Q1 : Quelle colonne de fact_budget permettrait de la joindre à dim_product ? (si aucune, expliquez pourquoi)
8. -- Q2 : Quelle mesure de fact_inventory_snapshot ne peut PAS être additionnée sur plusieurs dates ?
9. -- Q3 : Combien de dimensions (colonnes *_key) partagent fact_sales et fact_returns ?
10. SELECT DISTINCT category FROM fact_budget ORDER BY 1;
11. SELECT DISTINCT category FROM dim_product ORDER BY 1;
12. -- Les deux listes doivent être identiques (case-sensitive). Notez tout écart.
13. -- PIÈGE INTENTIONNEL – exécutez ceci et notez le résultat :
14. SELECT COUNT(*) FROM fact_sales JOIN fact_returns USING (product_key);
15. -- Comparez avec : SELECT COUNT(*) FROM fact_sales; et SELECT COUNT(*) FROM fact_returns;
16. -- Notez le ratio dans un commentaire. Vous allez résoudre ce problème en Exercice 2.

```

⚠ Écueils fréquents

- Q1 sans réponse : fact_budget n'a PAS de product_key — c'est intentionnel, pas un oubli
- fact_budget.category ne correspond pas à dim_product.category : vérifiez majuscules et espaces
- Table vide après make load : re-lancez make generate puis make load
- Piège ignoré : ne pas documenter le fan-out = ne pas comprendre le problème à résoudre

Exercice 1 — Bus Matrix

10 min

Construire la bus matrix à partir de vos propres requêtes de découverte — pas à partir du slide. C'est le livrable d'architecture le plus important.

```

1. -- Étape 1 : listez les colonnes de chaque table et identifiez les FKs (*_key)
2. PRAGMA table_info(fact_sales);
3. PRAGMA table_info(fact_returns);
4. PRAGMA table_info(fact_inventory_snapshot);
5. PRAGMA table_info(fact_budget);
6. -- Pour chaque table : quelles colonnes *_key apparaissent ? Notez-les avant de continuer.
7. -- Étape 2 : créez docs/bus-matrix.md à partir de VOS résultats (pas du slide).
8. -- Format : tableau markdown 5 dimensions × 4 tables. Cases vides = absence justifiée.
9. -- Étape 3 : vérification croisée avec le slide de la bus matrix – coïncide ?
10. -- Si votre matrice diffère du slide : revérifiez vos PRAGMA. Documentez toute divergence.
11. SELECT COUNT(DISTINCT product_key) FROM fact_sales;
12. SELECT COUNT(DISTINCT product_key) FROM fact_returns;
13. -- Les deux product_key sets pointent vers les mêmes entrées dans dim_product.
14. git add docs/bus-matrix.md && git commit -m 'S07 bus-matrix documented'

```

⚠ Écueils fréquents

- Copier le slide sans faire les PRAGMA : vous ne saurez pas si votre DB correspond
- Coche pour dim_customer dans fact_budget : fact_budget n'a pas de customer_key
- Oublier fact_inventory_snapshot dans la matrice
- Cases vides sans justification : expliquez pourquoi la dimension est absente

Exercice 2 — Drill-across ventes vs retours

35 min

Écrire un drill-across fonctionnel entre `fact_sales` et `fact_returns` via `dim_product` — en partant du problème observé en Exercice 0.

```
1. -- PARTIE A : tentative naïve (5 min) — commencez ici avant tout autre SQL
2. Creez sql/integration/s07-drill-across.sql
3. -- Écrivez vous-même la requête naïve :
4. -- SELECT p.category, SUM(s.line_total) AS rev, SUM(r.refund_amount) AS ref
5. -- FROM fact_sales s
6. -- JOIN fact_returns r USING (product_key)
7. -- JOIN dim_product p USING (product_key)
8. -- GROUP BY 1;
9. -- Comparez SUM(rev) obtenu avec SELECT SUM(line_total) FROM fact_sales
10. -- Notez le ratio d'inflation dans un commentaire : -- INFLATION OBSERVÉE : x.xx
11. -- PARTIE B : construction guidée (20 min)
12. -- Construisez deux CTEs séparées (pas de skeleton — description seulement) :
13. -- CTE 1 : agréger fact_sales via dim_product → grain category × mois
14. -- Colonnes cibles : category, mois, revenue (SUM line_total), units_sold (SUM quantity)
15. -- CTE 2 : agréger fact_returns via dim_product → même grain category × mois
16. -- Colonnes cibles : category, mois, total_refunds (SUM refund_amount), units_returned
17. -- Jointure finale : FULL OUTER JOIN sur (category, mois). Jamais INNER JOIN.
18. -- Utilisez COALESCE pour les colonnes de grain dans le SELECT final.
19. -- PARTIE C : réconciliation obligatoire (10 min)
20. -- Ajoutez ce commentaire avec le résultat observé :
21. -- SELECT SUM(revenue) FROM sales_agg; ⇒ [résultat]
22. -- SELECT SUM(line_total) FROM fact_sales; ⇒ [résultat]
23. -- Ces deux valeurs doivent être identiques. Si non : GROUP BY incomplet (ajoutez mois).
24. git add -A && git commit -m 'S07 sprint1 drill-across fonctionnel'
```

⚠ Écueils fréquents

- Sauter la Partie A : le fan-out doit être vécu avant d'être compris
- JOIN direct conservé comme solution finale : produit cartésien garanti
- INNER JOIN au lieu de FULL OUTER JOIN : perd les catégories avec 0 retours
- GROUP BY incomplet (oublier le mois) : totaux non réconciliables

Exercice 3 — Réel vs budget

30 min

Construire une vue `actual_vs_budget` au grain `category × store_key × mois`, avec `variance` et `pct_variance` — en commençant par découvrir le design de `fact_budget` avant d'écrire une seule ligne de SQL.

```
1. -- PARTIE A : anatomie de fact_budget (5 min)
2. Creez sql/integration/s07-actual-vs-budget.sql
3. DESCRIBE fact_budget;
4. -- Répondez dans un bloc -- OBSERVATIONS : en haut du fichier :
5. -- Q1 : Y a-t-il un product_key dans fact_budget ? Pourquoi ?
6. -- Q2 : Quel est le grain exact de fact_budget ? (colonnes qui forment la clé naturelle)
7. -- Q3 : Quelle colonne fait office de "produit" dans cette table ?
8. -- PARTIE B : tentative naïve (5 min)
9. -- Essayez cette jointure et observez le résultat :
10. -- SELECT * FROM fact_sales s JOIN fact_budget b ON s.store_key = b.store_key LIMIT 10;
11. -- Qu'arrive-t-il aux catégories ? Notez dans un commentaire pourquoi les résultats sont incorrects.
12. -- PARTIE C : construction guidée (15 min) — pas de skeleton fourni
13. -- CTE actual : agréger fact_sales via dim_product → category × store_key × mois
14. -- Le pont : fact_sales.product_key → dim_product.category = fact_budget.category
15. -- Colonnes cibles : category, store_key, mois, actual_revenue (SUM line_total)
16. -- FULL OUTER JOIN fact_budget sur le grain commun : (category, store_key, budget_month)
17. -- SELECT final : category, store_key, mois, actual_revenue, target_revenue, variance, pct_variance
18. -- variance = actual_revenue - target_revenue
19. -- pct_variance = ROUND(100.0 * variance / NULLIF(target_revenue, 0), 1)
20. -- PARTIE D : réconciliation obligatoire (5 min)
21. Creez sql/checks/s07-reconciliation.sql :
22. -- SELECT SUM(actual_revenue) FROM actual; ⇒ [résultat observé]
23. -- SELECT SUM(line_total) FROM fact_sales; ⇒ [résultat observé]
24. -- Ces deux valeurs doivent être identiques. Si non : GROUP BY incomplet ou dim_product manquant.
25. git add -A && git commit -m 'S07 sprint2 actual-vs-budget + reconciliation'
```

⚠ Écueils fréquents

- Sauter la Partie A : ne pas comprendre le design de `fact_budget` avant de coder
- Oublier `dim_product` dans la CTE `actual` : `fact_sales` n'a pas de colonne `category`
- INNER JOIN avec `fact_budget` : perd les catégories avec budget mais 0 ventes réelles
- Division sans NULLIF : erreur de division par zéro si `target_revenue = 0`
- Joindre sur `store_id` au lieu de `store_key` : `fact_budget` n'a pas de colonne `store_id`

Remise & évaluation

ARTÉFACTS REQUIS

- ☐ answers/S07_executive_brief.md
- ☐ db/nexamart.duckdb
- ☐ ai-usage.md

RÈGLE DE REMISE

 Before next session starts

CRITÈRES D'ÉVALUATION

%	Dimension	Accent de la séance
40%	Model quality	Bus matrix complète. Drill-across entre deux fact tables via une dimension conforme.
25%	Validation quality	Requête drill-across retourne un résultat cohérent entre les deux tables (même dimension de jointure).
20%	Executive justification	Brief répond à une question qui nécessite les deux tables. Énonce explicitement la dimension conforme utilisée.
10%	Process trace	Bus matrix commitée dans docs/bus-matrix.md. Décision de grain partagé documentée.
5%	Reproducibility	

EXIT TICKET

 Qu'est-ce que le CEO peut décider maintenant que votre modèle S07 est livré ?

DIVULGATION IA — AI-USAGE.MD

1

Outil utilisé
Nom, version, date d'accès

2

Ce qu'il m'a aidé à faire
Tâche précise — pas « écrire le travail »

3

Sources et chiffres vérifiés
Chaque affirmation → source primaire consultée

4

Ce que j'ai modifié ou rejeté
Corrections, reformulations, refus — et pourquoi

5

Déclaration de responsabilité
Je suis responsable de l'exactitude et du jugement — outil IA utilisé ou non.

Présent même si aucun outil IA utilisé · gabarit : [content/templates/ai-usage.md](#)

Vos outils & règles IA pour cette séance

Voici comment vous travaillez ce soir : ce qui est permis, ce qui doit être tracé, et la boîte à outils gratuite à votre disposition.

POLITIQUE IA (COURS)

COPILLOT REQUIS **VS CODE REQUIS** **DIVULGATION IA OBLIGATOIRE**

Chaque utilisation d'un assistant IA (Copilot, ChatGPT, Claude, etc.) doit être tracée dans `ai-usage.md` avec le prompt et la validation.

VOS OUTILS INDISPENSABLES

vscode

github

duckdb

mermaid

markdown

À DÉCOUVRIR EN DÉMO

bigquery sandbox

looker studio

POUR ALLER PLUS LOIN (OPTIONNEL)

supabase

cloudflare workers

cloudflare pages

POLITIQUE CLOUD

AUCUNE CARTE REQUISE

DONNÉES SYNTHÉTIQUES UNIQUEMENT

Voie par défaut : `local_duckdb`

POUR VOUS, CONCRÈTEMENT

- Avant S1 : vérifiez votre **GitHub Student Pack**, acceptez l'invitation Classroom.
- Chaque séance : ouvrez votre **Codespace** avant le début.
- Chaque séance : **commitez avant de partir**, même si c'est inachevé.
- Remplissez l'exit ticket avant 22 h.

Lectures recommandées



Kimball Group -- Conformed Dimensions

Dimensions partagées entre tables de faits pour le drill-across

<https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/dimensional-modeling-techniques/conformed-dimension/>



Kimball Group -- Enterprise Data Warehouse Bus Architecture

La bus matrix comme outil d'intégration entre tables de faits

<https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/kimball-data-warehouse-bus-architecture/>



DuckDB -- Multiple Result Sets

Syntaxe SQL pour les requêtes multi-tables et les CTEs

https://duckdb.org/docs/sql/query_syntax/select

Exit ticket

Répondez à la question ci-dessous **avant 22 h ce soir**. Votre réponse est individuelle et compte dans votre participation.

QUESTION DE RÉFLEXION

Qu'est-ce que le CEO peut décider maintenant que votre modèle S07 est livré ?

COMMENT RÉPONDRE

1. Sortez votre téléphone.
2. Scannez le code QR ou tapez l'adresse ci-dessous.
3. Connectez-vous avec votre courriel UdeS.
4. Rédigez votre réponse et cliquez *Soumettre*.

<https://profos-dhowwtjefa-nn.a.run.app/exit-ticket/GIS805-07>

